01d — Math Operations & Built-in Functions (beginner-friendly)

These are the tools you **compute** with: the math symbols and the ready-made functions Python gives you for free. Every one listed here shows up on the exam. For each: what it does + a **scenario** where you'd reach for it.

Part A — Math operators

```
a + b     # add
a - b     # subtract
a * b     # multiply
a / b     # divide → ALWAYS a float:  7 / 2 = 3.5
a // b    # integer divide (floor) → drops the decimal:  7 // 2 = 3
a % b     # remainder (modulo):  7 % 2 = 1
a ** b    # power:  2 ** 10 = 1024
```

The two exam-gold operators: `//` and `%`

`//` and `%` are the most useful operators on the whole exam. Memorize these uses:

I want...	Use	Example
the last digit of a number	`n % 10`	`1234 % 10` = 4
remove the last digit	`n // 10`	`1234 // 10` = 123
is n even?	`n % 2 == 0`	
is n divisible by k?	`n % k == 0`	
wrap around a list of size k (cycle)	`i % k`	index `7 % 3` = 1
split into groups of k	`total // k`	17 people, groups of 5 → `17 // 5` = 3 full groups

> **`/` vs `//`:** `/` gives a decimal (`3.5`), `//` gives a whole number (3). On this exam you almost always want `//`. Using `/` by accident (getting `3.5` where you wanted 3) is a common bug.

Scenario for `%` (remainder): anything about digits, even/odd, cycles, "every k-th".
Scenario for `//` (floor divide): "how many whole groups", removing digits, midpoints.

Part B — Number built-in functions

```
abs(-5)        # 5      — distance from zero (always positive)
round(3.7)     # 4      — nearest whole number
round(3.14159, 2)  # 3.14 — round to 2 decimals
min(4, 7, 1)   # 1      — smallest
max(4, 7, 1)   # 7      — largest
pow(2, 10)     # 1024   — same as 2 ** 10
```

Scenarios:

- `abs` → "difference between two numbers regardless of order": `abs(a - b)`.
- `min` / `max` on two values → clamp/limit: `max(0, x)` keeps x from going negative.

Part C — Converting between types

```
int("42")     # 42     — string → integer
int(3.9)      # 3      — float → integer (CUTS the decimal, doesn't
str(42)       # "42"   — number → string  (needed to walk digits, 
float("3.5")  # 3.5    — string → float
```

Scenarios:

- `str(n)` → to loop over a number's digits: `for d in str(n)`.
- `int(d)` → turn a digit character back into a number.
- `int(x)` → chop a decimal (careful: `int(3.9)` is 3, not 4 — use `round` if you want nearest).

Part D — Collection built-in functions (used constantly)

```
len(x)              # how many items (works on list, string, dict, s
sum(nums)           # add up all numbers in a list
min(nums)           # smallest item
max(nums)           # largest item
sorted(nums)        # NEW sorted list (original untouched)
sorted(nums, reverse=True)   # biggest → smallest
```

Scenarios:

- `len` → loop bounds, "how long is it", checking emptiness.
- `sum` → totals, averages (`sum(nums) / len(nums)`).
- `min`/`max` → "the best/worst", also with `key=` (below).
- `sorted` → "put in order first, then it's easy" (file 05).

> **`sorted()` vs `.sort()`:** `sorted(nums)` returns a NEW list, leaves the original alone. `nums.sort()` changes the list in place and returns `None`. Don't write `a = nums.sort()` — that sets `a` to `None`.

The `key=` trick (works in `max`, `min`, `sorted`)

`key=` says "don't compare the items directly — compare THIS measurement of them":

```
max(words, key=len)          # the LONGEST word (compare by length)
min(words, key=len)          # the SHORTEST word
sorted(words, key=len)       # order words short → long
sorted(nums, key=abs)        # order by distance from zero
sorted(people, key=lambda p: p[1])   # sort by the 2nd field of eacl
```

Scenario: "longest/shortest ___", "sort by score/age/length", "biggest by some measure".

Part E — Loop helper functions

```
range(5)              # 0,1,2,3,4      — numbers to loop over
range(2, 8, 2)        # 2,4,6          — start, stop, step
enumerate(items)      # gives (index, item) pairs
zip(a, b)             # pairs items of two lists together
list(range(5))        # [0,1,2,3,4]    — turn a range into a real l
```

(Full loop usage is in file 01b.) **Scenarios:** `range` → repeat/count; `enumerate` → need the position too; `zip` → two lists side by side.

Part F — Making collections

```
list("cat")        # ['c','a','t']   — string → list of chars
"".join(chars)     # 'cat'           — list of strings → one string
list(range(5))     # [0,1,2,3,4]
[0] * 5            # [0,0,0,0,0]      — a filled list
set(nums)          # unique values, as a set
dict()  {}         # empty dictionary
```

Scenarios:

- `list(s)` → to edit a string (strings are immutable): list it, change it, join it back.
- `"".join(...)` → build a string from pieces at the end of a loop.
- `[0] * n` → a counting array (e.g., count of each letter).
- `set(nums)` → remove duplicates / fast membership.

Part G — Counter and other handy helpers

```
from collections import Counter
Counter("mississippi")          # {'i':4,'s':4,'p':2,'m':1} — counts
Counter(nums).most_common(1)    # [(value, count)] of the most frequ

# Useful checks that return True/False:
any(x > 10 for x in nums)       # is ANY item > 10?
all(x > 0  for x in nums)       # are ALL items > 0?
```

Scenarios:

- `Counter` → "how many of each", "most frequent" (file 03).
- `any` → "does at least one match?"
- `all` → "do they all match?"

Quick "which tool?" reference

I want to...	Use
last digit / even-odd / divisible / cycle	`%`
remove last digit / how many groups	`//`
distance between two numbers	`abs(a - b)`

number → text (to walk digits)	`str(n)`
text → number	`int(s)`
how many items	`len(x)`
total of a list	`sum(nums)`
biggest / smallest	`max` / `min` (add `key=` to measure by something)
put in order	`sorted(nums)`
count how many of each	`Counter(items)`
does any / do all match	`any(...)` / `all(...)`
unique values / fast membership	`set(nums)`
build a string from pieces	`"".join(list)`